INTERNET OF THINGS (IoT)

PRACTICAL MANUAL

All 10 practicals compiled into one PDF

Programs, connections, algorithms, references and sample outputs from the uploaded practical manual.

Content is organized for easier copying. Technical details and code are transcribed from the provided manual; check hardware and library compatibility before running.

1. Familiarization with Raspberry Pi and necessary software installation

1. Download and install Raspberry Pi Imager 2.0. Visit raspberrypi.com/software, download Raspberry Pi Imager for your operating system, install it, and open it.

2. Prepare the SD card. Insert a microSD card (minimum 32 GB; Class 10 or faster recommended).

3. Select device and operating system. Choose your Raspberry Pi model, then select Raspberry Pi OS. Use 64-bit for Raspberry Pi 4/5 where appropriate, or 32-bit for compatibility with older models.

4. Select storage carefully. Writing the image erases all data on the selected card.

5. Configure location, language, time zone, hostname, username and password, Wi-Fi, and optional SSH/Raspberry Pi Connect access.

6. Review settings and click Write. Wait for the image to be written and verified. Insert the card into the Raspberry Pi and boot it.

2. Python program on Raspberry Pi: word and character count

Program:

```
text = input("Enter a string: ")

word_count = len(text.split())
char_count = len(text)

print("Number of Words:", word_count)
print("Number of Characters:", char_count)
```

Sample output:

```
Enter a string: Welcome to Raspberry Pi
Number of Words: 4
Number of Characters: 23
```

Run on Raspberry Pi: Open Terminal, create a file with `nano word_char_count.py`, paste the code, save with Ctrl+X, Y, Enter, then run `python3 word_char_count.py`.

3. Exercise on working principle of Raspberry Pi Pico with 40-pin interface

Reference: https://wokwi.com/projects/393048863616453633

Program:

```
void setup() {
  pinMode(15, INPUT);
  Serial1.begin(115200);
  Serial1.println("Hello, Raspberry Pi Pico!");
}

void loop() {
  int pir = digitalRead(15);
  if (pir == HIGH) {
    Serial1.println("MOVEMENT DECTECTED");
    delay(500);
  } else {
    Serial1.println("NO MOVEMENT DECTECTED");
    delay(500);
  }
}
```

4. Interface Bluetooth with Raspberry Pi Pico to turn LED ON/OFF from smartphone

Note: Physical device required. Reference: https://iot-amrt.vlabs.ac.in/exp/bluetooth-led-control-pi/index.html

Connections: HC-05 VCC → VBUS (5V); TXD → GP0; RXD → GP1; GND → Pico GND. Connect LED positive terminal through a resistor to GP19 and LED negative terminal to GND. Pair the module with a smartphone and send ON/OFF commands.

Components: Raspberry Pi Pico, HC-05, resistor, LED, smartphone.

Program (MicroPython):

```
from machine import Pin, UART

uart = UART(0, 9600)
led = Pin(19, Pin.OUT)

while True:
    if uart.any():
        command = uart.readline()
        if command == b'ON':
            led.high()
            print("LED ON")
        elif command == b'OFF':
            led.low()
            print("LED OFF")
```

5. Connect to available Wi-Fi using Arduino

Note: Physical device / simulator required. Reference: https://wokwi.com/projects/356552645200576513

Program:

```
#include <WiFi.h>

#define ssid "Wokwi-GUEST"
#define password ""

void initWiFi() {
  WiFi.mode(WIFI_STA);
  WiFi.begin(ssid, password);
  Serial.print("Connecting to WiFi ..");
  while (WiFi.status() != WL_CONNECTED) {
    Serial.println(WiFi.status());
    Serial.print('.');
    delay(1000);
  }
  Serial.println("Connected");
  Serial.println(WiFi.status());
  Serial.println(WiFi.localIP());
  Serial.print("RSSI: ");
  Serial.println(WiFi.RSSI());
}

void setup() {
  Serial.begin(115200);
  initWiFi();
}

void loop() {
  // Code to run repeatedly can be added here.
}
```

6. Raspberry Pi program to retrieve temperature and humidity from ThingSpeak cloud

Reference: https://wokwi.com/projects/469089414257926145

Program (MicroPython):

```
import network, urequests, time

wlan = network.WLAN(network.STA_IF)
wlan.active(True)
wlan.connect("Wokwi-GUEST", "")
while not wlan.isconnected():
    time.sleep(0.5)
print("WiFi connected\n")
```

```
CHANNEL_ID = "12397"
READ_KEY = ""
TEMP_FIELD = "field4"
HUM_FIELD = "field3"

URL = "https://api.thingspeak.com/channels/" + CHANNEL_ID + "/feeds/last.json"
if READ_KEY:
    URL += "?api_key=" + READ_KEY

while True:
    try:
        r = urequests.get(URL)
        data = r.json()
        r.close()
        if isinstance(data, dict):
            print("Time :", data.get("created_at"))
            print("Temp :", data.get(TEMP_FIELD))
            print("Humidity:", data.get(HUM_FIELD))
            print("-" * 30)
        else:
            print("Bad channel ID / key, server said:", data)
    except Exception as e:
        print("Network error:", e)
    time.sleep(20)
```

7. Raspberry Pi program to publish temperature data to an MQTT broker

Reference: https://wokwi.com/projects/322577683855704658

Program (MicroPython):

```
import network
import time
from machine import Pin
import dht
import ujson
from umqtt.simple import MQTTClient

MQTT_CLIENT_ID = "micropython-weather-demo"
MQTT_BROKER = "broker.mqttdashboard.com"
MQTT_USER = ""
MQTT_PASSWORD = ""
MQTT_TOPIC = "wokwi-weather"

sensor = dht.DHT22(Pin(15))

print("Connecting to WiFi", end="")
sta_if = network.WLAN(network.STA_IF)
sta_if.active(True)
sta_if.connect("Wokwi-GUEST", "")
while not sta_if.isconnected():
    print(".", end="")
    time.sleep(0.1)
print(" Connected!")

print("Connecting to MQTT server... ", end="")
client = MQTTClient(
    MQTT_CLIENT_ID, MQTT_BROKER,
    user=MQTT_USER, password=MQTT_PASSWORD
)
client.connect()
print("Connected!")

prev_weather = ""
while True:
    print("Measuring weather conditions... ", end="")
    sensor.measure()
    message = ujson.dumps({
        "temp": sensor.temperature(),
        "humidity": sensor.humidity(),
    })
    if message != prev_weather:
        print("Updated!")
        print("Reporting to MQTT topic {}: {}".format(MQTT_TOPIC, message))
        client.publish(MQTT_TOPIC, message)
        prev_weather = message
    else:
        print("No change")
```

```
    time.sleep(1)
```

8. Connect Raspberry Pi with an existing component: passive buzzer

Objective: Test a passive buzzer to play an alarm sound at one-second intervals.

Reference: https://wokwi.com/projects/357627199664450561

Connections: Passive buzzer GND → Raspberry Pi Pico GND; positive pin → GPIO 15.

Program (MicroPython):

```
from picozero import Speaker
from time import sleep

speaker = Speaker(15)

while True:
    speaker.on()
    sleep(1)
    speaker.off()
    sleep(1)
```

9. IoT-based DC motor speed control using Arduino/Raspberry Pi

Reference: https://wokwi.com/projects/462024572818446337

Program (MicroPython for Raspberry Pi Pico):

```
from machine import Pin, PWM, ADC
import time

dc_motor = PWM(Pin(15))
dc_motor.freq(1000)

servo = PWM(Pin(16))
servo.freq(50)

pot = ADC(26)

def set_servo_duty(angle):
    duty = int(((angle / 180) * 6554) + 1638)
    servo.duty_u16(duty)

current_angle = 0
step_direction = 1

while True:
    pot_val = pot.read_u16()
    dc_motor.duty_u16(pot_val)

    speed_delay = (65535 - pot_val) / 1000000 + 0.001

    if pot_val > 1000:
        current_angle += step_direction
        if current_angle >= 180 or current_angle <= 0:
            step_direction *= -1
        set_servo_duty(current_angle)
        speed_percent = int((pot_val / 65535) * 100)
        print(f"DC Speed: {speed_percent}% | Servo Speed: {speed_percent}%", end="\r")
    else:
        print("Motors Stopped...        ", end="\r")

    time.sleep(speed_delay)
```

10. IoT data logging using BeagleBone Black and ThingSpeak

Reference: https://wokwi.com/projects/460072555056502785

Hardware required: BeagleBone Black (1), USB cable (1), breadboard (1), LDR (1), 10 kΩ resistor (1), connecting wires, PC/laptop with Internet.

Software required: Debian Linux on BeagleBone Black, Python, Adafruit_BBIO library, Internet connection, ThingSpeak account, SSH terminal (PuTTY).

Algorithm: 1. Start BeagleBone Black and connect it to the Internet. 2. Create a ThingSpeak account/channel and note the Write API Key. 3. Connect the LDR circuit to an analog input. 4. Initialize ADC using Adafruit_BBIO. 5. Read the sensor value and convert it to voltage. 6. Send the voltage to ThingSpeak using HTTP POST. 7. Wait 15–20 seconds and repeat. 8. Observe the real-time graph.

Program (as provided in the source manual):

```
import Adafruit_BBIO.ADC as ADC
import time
import httplib, urllib

sensor_pin = "P9_40"
ADC.setup()

while True:
    reading = ADC.read(sensor_pin)
    voltage = reading * 1.8

    params = urllib.urlencode({
        'field1': voltage,
        'key': 'YOUR_WRITE_API_KEY'
    })
    headers = {
        "Content-type": "application/x-www-form-urlencoded",
        "Accept": "text/plain"
    }

    conn = httplib.HTTPConnection("api.thingspeak.com:80")
    conn.request("POST", "/update", params, headers)
    response = conn.getresponse()

    print("ADC Reading :", reading)
    print("Voltage :", voltage)
    time.sleep(16)
```

Replace YOUR_WRITE_API_KEY with the Write API Key from your ThingSpeak channel.

Working: The LDR senses light intensity; voltage changes with light. The BeagleBone Black ADC reads the voltage, Python converts it, and the program uploads it to ThingSpeak. ThingSpeak stores the readings and generates graphs that update every 15–20 seconds.

Sample output: ADC Reading: 0.412; Voltage: 0.7416 V; HTTP Response: 200 OK; Data uploaded successfully.

Result: Analog sensor data was acquired using BeagleBone Black and uploaded to ThingSpeak for real-time graph visualization.